

Instrument Registry / Configuration Classes

Module Goal: Provide a centralised, thread-safe store of all tradable instruments (symbols) and venue-specific configuration. This is the single source of truth for symbol rules, validation, routing decisions, and future venue differences.

Instrument

```
{ struct }  
+ std::string Symbol // Example: "BTC-USDT", "ETH-BTC", "SOL-USDC"  
+ std::string baseAsset // Example: "BTC", "ETH"  
+ std::string quoteAsset // Example: "USDT", "BTC"  
+ double tickSize // minimum price increment (eg 0.01, 0.1, 0.0001)  
+ double minOrderQty // minimum quantity allowed in an order  
+ double maxOrderQty // maximum quantity per order (optional soft limit)  
+ double qtyStep // quantity increment / step size (eg 0.001 BTC)  
+ int pricePrecision // Number of decimal places for price display / rounding  
+ int qtyPrecision // Number of decimal places for quantity  
+ bool isActive = true // whether trading is currently enabled for this symbol  
+ std::string venue // If multi-venue from the start: "Binance", "Bybit", etc
```

Data Structure representing one tradable symbol / pair.

Used as value type inside the registry map.

Instrument Registry

```
- std::unordered_map<std::string, Instrument> instruments // Key = Symbol (in uppercase) // Value = Instrument  
- mutable std::shared_mutex mMutex // Allows multiple concurrent readers + exclusive writers  
  
+ void loadFromJson(const std::string & configPath) // Load instruments from JSON file at startup  
+ void addInstrument(const Instrument & instrument) // Add or update a symbol (used by admin or config reload)  
+ std::vector<Instrument> getInstrument(const std::string & symbol) const  
+ bool containsSymbol(const std::string & symbol) const  
+ bool isActive(const std::string & symbol) const  
+ bool validateSymbol(const std::string & symbol) const // returns true if symbol exists and is active  
+ bool isPriceValid(const std::string & symbol, double price) const // checks: price > 0, price % tickSize == 0 (within epsilon)  
+ bool isQuantityValid(const std::string & symbol, double qty) const // checks: qty >= minOrderQty, qty <= maxOrderQty, qty % qtyStep == 0  
+ std::vector<std::string> getAllActiveSymbols() const // useful for feed subscription, market data, etc
```

Central lookup and validation service for all instruments.

Validation Checks.

Which symbols to track

Symbol → Venue or book lookup.

Which symbols to subscribe to.

Choose venue based on fees / latency

Concurrency / Thread Safety Notes

- Read-heavy workload → `std::shared_mutex`
- All `const` methods use shared lock (`mutex.lock_shared()`)
- Write methods use exclusive lock (`mutex.lock()`)
- In V1: Loading happens at startup → Very few writes after initialisation.
- Avoid copying the entire map - return by value or `const` ref where possible

Order Validator

Order Router

Market Data Aggregator

Feed Connector / Feed Dispatcher

Connect to the correct websocket URL.

(Optional in minimal V1 - include if planning multi-venue support soon)

Venue Config

```
- std::string venueName // "Binance", "Bybit", "Coinbase", etc  
- double makerFee = 0.0002  
- double takerFee = 0.0004  
- uint32_t maxLatencyMs = 150 // estimated round-trip latency (for routing decisions)  
- std::string wsUrl // websocket endpoint for market data / user orders  
- std::unordered_map<std::string, std::string> extraParams // API key, secret, custom headers, etc (Avoid storing secrets in production code)  
  
+ double getEffectiveFee(bool isMaker) const  
+ bool isVenueSupported(const std::string & symbol) const
```

owned or referenced by InstrumentRegistry (if per-symbol venue)

Load from config or secure vault.

Per-venue settings can be extended later for fees, API keys, rate limits, etc.

In crypto exchanges, these values come from /exchangeInfo or /symbols endpoints.